

Josh Williams

SQL QA Portfolio

Data Validation - Defect Investigation - Reporting Verification - Test Design

This portfolio presents selected SQL examples framed as Quality Assurance validation work. Each project shows how SQL can be used to test business rules, verify dashboards, investigate data defects, and support regression testing.

Professional Summary

This portfolio demonstrates practical SQL capability for entry-level QA Analyst work. The selected queries emphasize real QA responsibilities: validating report totals, checking relationships between tables, confirming business rules, testing ranking logic, and investigating potential data defects.

Portfolio Area	Representative Skills
Reporting Validation	COUNT FILTER, GROUP BY, date ranges, KPI verification
Data Integrity Testing	LEFT JOIN, multi-table joins, relationship checks
Business Rule Validation	CTEs, aggregate comparisons, thresholds
Window Function Testing	ROW_NUMBER, LEAD, LAG, running totals
QA Investigation	Expected results, possible defects, retesting logic

Table of Contents

- QA-SQL-001 - User Login Frequency Distribution
- QA-SQL-002 - User Video View Distribution Analysis
- QA-SQL-003 - Employee Relationship Data Validation
- QA-SQL-004 - Top 3 Baby Names by Gender
- QA-SQL-005 - Sales Running Total and Running Average
- QA-SQL-006 - Daily Sales Previous-Day and Next-Day Comparison
- QA-SQL-007 - Category Pricing Benchmark Analysis
- QA-SQL-008 - Chicken Menu Item Category Price Validation
- QA-SQL-009 - Quarterly Video View Validation
- QA-SQL-010 - Restaurant Half-Year Order Comparison
- QA-SQL-011 - Customer Transaction Validation
- QA-SQL-012 - Menu Category Price Threshold Validation

Joins, Subqueries, and User Activity Validation

QA-SQL-001: User Login Frequency Distribution

Business Problem: The business needs to understand how many users logged in 0, 1, 2, or more times during 2022, including users with no login activity.

Test Objective: Validate that the user activity report correctly counts logins per user and groups users into login-count buckets.

SQL Query:

```
SELECT
    login_count AS login_bucket,
    COUNT(user_id) AS number_of_users
FROM (
    SELECT
        u.user_id,
        COUNT(l.login_id) AS login_count
    FROM users u
    LEFT JOIN logins l
        ON l.user_id = u.user_id
        AND login_date BETWEEN '2022-01-01' AND '2022-12-31'
    GROUP BY u.user_id
) number_of_logins
GROUP BY login_count
ORDER BY login_count;
```

Expected Result: Each user appears in exactly one login bucket. Users with no matching login records should appear in the 0-login bucket.

QA Use Case: Validates user activity dashboards, engagement reports, and inactive-user reporting.

Possible Defects Identified: Incorrect join logic, missing zero-activity users, inflated counts from duplicate login rows, or incorrect date filtering.

Skills Demonstrated: LEFT JOIN, subquery, COUNT(), GROUP BY, date filtering, bucketed reporting, data validation.

QA-SQL-002: User Video View Distribution Analysis

Business Problem: A product team wants to know how video views are distributed across the user base for 2023.

Test Objective: Verify that users are grouped by their number of video views and that users with zero views are still included.

SQL Query:

```
SELECT
    view_count AS view_bucket,
    COUNT(user_id) AS user_count
FROM (
    SELECT
        u.user_id,
        COUNT(v.view_id) AS view_count
    FROM users u
    LEFT JOIN views v
        ON v.user_id = u.user_id
        AND v.view_date BETWEEN '2023-01-01' AND '2023-12-31'
    GROUP BY u.user_id
) user_view_count
GROUP BY view_count
ORDER BY view_count;
```

Expected Result: Every user appears in one view bucket, including a 0-view bucket for users with no views.

QA Use Case: Validates engagement dashboards, analytics reports, and year-end user activity summaries.

Possible Defects Identified: Users disappearing from reports, duplicate views inflating totals, wrong date boundaries, or incorrect grouping logic.

Skills Demonstrated: LEFT JOIN, subquery, aggregation, GROUP BY, ORDER BY, reporting validation.

Relational Data Integrity and Multi-Table Joins

QA-SQL-003: Employee Relationship Data Validation

Business Problem: An HR report needs to display employees with their department, manager, and project assignment information.

Test Objective: Verify that employee records connect correctly across department, manager, and project tables.

SQL Query:

```
SELECT
    e.emp_name,
    m.manager_name,
    d.department_name,
    p.project_name
FROM employees e
LEFT JOIN department d
    ON d.id = e.dep_id
JOIN manager m
    ON m.manager_id = e.manager_id
LEFT JOIN projects p
    ON p.team_member_id = e.emp_id;
```

Expected Result: Employees should appear with the correct department and manager. Employees without projects should still appear because of the LEFT JOIN.

QA Use Case: Validates HR dashboards, employee profile pages, org charts, and project assignment reports.

Possible Defects Identified: Incorrect foreign keys, employees dropped by wrong join type, missing department data, invalid manager relationships, or project mismatches.

Skills Demonstrated: Multiple joins, LEFT JOIN, INNER JOIN, foreign key validation, relational data testing.

Window Functions and Ranking Validation

QA-SQL-004: Top 3 Baby Names by Gender

Business Problem: A reporting dashboard needs to display the three most popular baby names for each gender.

Test Objective: Validate that ranking resets within each gender and returns only the top three names per group.

SQL Query:

```
SELECT *
FROM (
    SELECT
        gender,
        name,
        total,
        ROW_NUMBER() OVER (
            PARTITION BY gender
            ORDER BY total DESC
        ) AS popularity
    FROM baby_names
) AS pop
WHERE popularity <= 3;
```

Expected Result: Each gender should return no more than three rows, ranked by highest total value.

QA Use Case: Validates top-N reports, leaderboards, category-specific rankings, and reporting display logic.

Possible Defects Identified: Ranking not resetting by gender, wrong sort order, too many/few results, or dashboard ranking mismatch.

Skills Demonstrated: ROW_NUMBER(), PARTITION BY, subquery, window functions, top-N analysis.

Window Functions and Trend Reporting

QA-SQL-005: Sales Running Total and Running Average

Business Problem: Sales leadership needs to track each employee's cumulative sales and running average over time.

Test Objective: Verify cumulative sales calculations and running average values by employee.

SQL Query:

```
SELECT
    *,
    SUM(sale_amount) OVER (
        PARTITION BY sales_employee_id
        ORDER BY sale_date
    ) AS running_total,
    AVG(sale_amount) OVER (
        PARTITION BY sales_employee_id
        ORDER BY sale_date
    ) AS running_avg
FROM sales
ORDER BY sale_date;
```

Expected Result: Running totals and averages should calculate separately for each employee and progress in sale-date order.

QA Use Case: Validates performance dashboards, trend reports, and cumulative KPI calculations.

Possible Defects Identified: Totals mixing employees, incorrect ordering, running average errors, or dashboard values not matching source data.

Skills Demonstrated: SUM() OVER, AVG() OVER, PARTITION BY, ORDER BY, running metrics, trend validation.

QA-SQL-006: Daily Sales Previous-Day and Next-Day Comparison

Business Problem: A sales report needs to compare each day's total sales to the previous and next day.

Test Objective: Validate daily totals and adjacent-day comparison logic.

SQL Query:

```
WITH daily AS (
    SELECT
        sale_date,
        SUM(sale_amount) AS daily_total
    FROM sales
    GROUP BY 1
)
SELECT
    *,
    LEAD(daily_total, 1) OVER (ORDER BY sale_date) AS next_day_total,
    LAG(daily_total, 1) OVER (ORDER BY sale_date) AS previous_day_total
FROM daily
ORDER BY 1;
```

Expected Result: Each row should show the current daily total, the next day total, and the previous day total when available.

QA Use Case: Validates day-over-day dashboards, anomaly checks, and trend comparison reports.

Possible Defects Identified: Incorrect daily totals, wrong date ordering, duplicated daily rows, or previous/next-day values shifted incorrectly.

Skills Demonstrated: CTE, SUM(), GROUP BY, LEAD(), LAG(), window functions, time-series validation.

CTEs, Aggregates, and Business Rule Validation

QA-SQL-007: Category Pricing Benchmark Analysis

Business Problem: Management wants to identify how many menu categories have a maximum price below the average maximum category price.

Test Objective: Validate category-level pricing benchmarks against a dynamic average.

SQL Query:

```
WITH mp AS (  
  SELECT  
    category,  
    MAX(price) AS max_price  
  FROM menu_items  
  GROUP BY category  
)  
SELECT COUNT(*)  
FROM mp  
WHERE max_price < (  
  SELECT AVG(max_price)  
  FROM mp  
)  
);
```

Expected Result: The query should return the count of categories whose maximum item price is below the average of all category maximums.

QA Use Case: Validates executive pricing dashboards, benchmark reports, and category-level business logic.

Possible Defects Identified: Wrong category grouping, incorrect maximum price, average calculation errors, missing categories, or KPI mismatches.

Skills Demonstrated: CTE, MAX(), AVG(), nested subquery, aggregate comparison, business rule testing.

CTEs, Joins, and Search Result Validation

QA-SQL-008: Chicken Menu Item Category Price Validation

Business Problem: A menu platform needs to find chicken-related items and compare each item to the maximum price in its category.

Test Objective: Validate menu search results and category-level price context for chicken items.

SQL Query:

```
WITH ci AS (  
    SELECT *  
    FROM menu_items  
    WHERE item_name LIKE '%chicken%'  
)  
mp AS (  
    SELECT  
        category,  
        MAX(price) AS max_price  
    FROM menu_items  
    GROUP BY category  
)  
SELECT *  
FROM ci  
LEFT JOIN mp  
    ON ci.category = mp.category;
```

Expected Result: All chicken-related menu items should appear with their category maximum price when a category match exists.

QA Use Case: Validates search results, category mapping, menu reporting, and price comparison features.

Possible Defects Identified: Missing search results, incorrect category assignments, price aggregation errors, or category join mismatches.

Skills Demonstrated: Multiple CTEs, LIKE, LEFT JOIN, MAX(), GROUP BY, search-result validation.

Time-Based Reporting and KPI Verification

QA-SQL-009: Quarterly Video View Validation

Business Problem: A video analytics dashboard reports user views by quarter for 2023.

Test Objective: Verify that quarterly view counts are calculated using the correct date ranges and grouped correctly by user.

SQL Query:

```
SELECT
    user_id,
    COUNT(*) FILTER (WHERE view_date BETWEEN '2023-01-01' AND '2023-03-31') AS q1_views,
    COUNT(*) FILTER (WHERE view_date BETWEEN '2023-04-01' AND '2023-06-30') AS q2_views,
    COUNT(*) FILTER (WHERE view_date BETWEEN '2023-07-01' AND '2023-09-30') AS q3_views,
    COUNT(*) FILTER (WHERE view_date BETWEEN '2023-10-01' AND '2023-12-31') AS q4_views
FROM video_views
WHERE EXTRACT(YEAR FROM view_date) = 2023
GROUP BY 1;
```

Expected Result: Each user should have separate counts for Q1, Q2, Q3, and Q4 based on 2023 view dates.

QA Use Case: Validates quarterly dashboards, user analytics, and KPI reporting logic.

Possible Defects Identified: Wrong quarter boundaries, records outside 2023 included, incorrect grouping, or dashboard totals not matching source data.

Skills Demonstrated: COUNT() FILTER, EXTRACT(), BETWEEN, GROUP BY, quarterly reporting validation.

QA-SQL-010: Restaurant Half-Year Order Comparison

Business Problem: A restaurant platform compares order volume between the first and second halves of 2023.

Test Objective: Validate half-year order counts by restaurant.

SQL Query:

```
SELECT
    restaurant_id,
    COUNT(*) FILTER (WHERE order_date BETWEEN '2023-01-01' AND '2023-06-30') AS first_half_orders,
    COUNT(*) FILTER (WHERE order_date BETWEEN '2023-07-01' AND '2023-12-31') AS second_half_orders
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2023
GROUP BY 1;
```

Expected Result: Each restaurant should show separate order counts for January-June and July-December 2023.

QA Use Case: Validates restaurant KPI dashboards, trend reports, and period-over-period comparisons.

Possible Defects Identified: Incorrect period boundaries, missing restaurants, duplicated orders, or wrong date filtering.

Skills Demonstrated: COUNT() FILTER, EXTRACT(), date filtering, GROUP BY, KPI validation.

Relationship Validation and Active-Customer Reporting

QA-SQL-011: Customer Transaction Validation

Business Problem: The business wants to identify customers who have at least one transaction.

Test Objective: Verify that transaction-based customer reports only include customers with valid transactions.

SQL Query:

```
SELECT
    c.first_name,
    c.last_name
FROM customers c
WHERE c.customer_id IN (
    SELECT DISTINCT customer_id
    FROM transactions
    WHERE customer_id IS NOT NULL
);
```

Expected Result: Every returned customer should have at least one transaction record, and NULL transaction customer IDs should be ignored.

QA Use Case: Validates active-customer dashboards, loyalty eligibility rules, transaction history reports, and customer segmentation.

Possible Defects Identified: Customers shown without transactions, orphaned transaction records, NULL foreign keys, or missing customer activity.

Skills Demonstrated: Subquery, IN, DISTINCT, NULL filtering, relationship validation, business rule verification.

CTEs, Aggregates, and Business Rule Validation

QA-SQL-012: Menu Category Price Threshold Validation

Business Problem: A restaurant platform needs to know how many categories have no item priced at \$15 or higher.

Test Objective: Validate category pricing rules and affordability filters.

SQL Query:

```
WITH mp AS (  
  SELECT  
    category,  
    MAX(price) AS max_price  
  FROM menu_items  
  GROUP BY category  
)  
SELECT COUNT(*)  
FROM mp  
WHERE max_price < 15;
```

Expected Result: The query should return the number of categories where the highest-priced item is less than 15.

QA Use Case: Validates pricing filters, category reports, promotional eligibility, and affordability rules.

Possible Defects Identified: Incorrect threshold logic, missing menu items, category grouping errors, or pricing data issues.

Skills Demonstrated: CTE, MAX(), GROUP BY, COUNT(), threshold validation, business rule testing.

End-to-End QA Investigation Project

Project: Video Analytics and User Engagement Validation

Scenario: A product dashboard reports quarterly views and user engagement buckets. QA needs to verify that the displayed metrics match the source database and that inactive users are not excluded from reporting.

Approach: 1) Validate quarterly view calculations. 2) Use LEFT JOIN logic to include users with zero activity. 3) Compare dashboard totals against SQL results. 4) Investigate discrepancies caused by date boundaries, duplicates, or missing relationships. 5) Retest after fixes.

Queries Used: QA-SQL-001, QA-SQL-002, and QA-SQL-009.

Outcome: Demonstrates a QA workflow that moves beyond writing queries into validating requirements, investigating defects, and supporting regression testing.

Recruiter-Friendly Skills Matrix

SQL Skill	Shown In Portfolio
SELECT / WHERE / ORDER BY	All examples
GROUP BY and Aggregations	QA-SQL-001, 002, 007, 009, 010, 012
LEFT JOIN and Multi-Table Joins	QA-SQL-001, 002, 003, 008
Subqueries	QA-SQL-001, 002, 004, 007, 011
CTEs	QA-SQL-006, 007, 008, 012
Window Functions	QA-SQL-004, 005, 006
Date Filtering	QA-SQL-001, 002, 009, 010
Data Validation	All examples
Defect Investigation	All QA use cases
Regression Testing Mindset	End-to-end QA project